

CS1501程序设计思想与方法C++ 课程笔记

uu-matter543

2023-2024-1

目录

1 绪论	7
1.1 程序设计概述	7
1.2 计算机组成	7
1.2.1 计算机硬件	7
1.2.2 计算机软件	7
1.3 程序设计语言	8
1.3.1 机器语言	8
1.3.2 汇编语言	8
1.3.3 高级语言	8
1.4 程序设计过程	8
1.4.1 算法设计	8
1.4.2 编程	8
1.4.3 编译与链接	9
1.4.4 调试与维护	9
2 程序基本结构	10
2.1 程序的基本结构	10
2.1.1 注释	10
2.1.2 预编译指令	10
2.1.3 名字空间	11
2.1.4 主程序	11
2.2 变量与常量	11
2.2.1 变量定义	12
2.2.2 数据类型	12
2.2.3 常量与符号常量	14

2.3	输入与输出	16
2.3.1	输入	16
2.3.2	输出	16
2.4	算术运算	17
2.4.1	算术表达式	17
2.4.2	不同类型数据的运算	17
2.4.3	强制类型转换	17
2.4.4	数学函数库	17
2.5	赋值运算	18
2.5.1	赋值表达式	18
2.5.2	多重赋值	19
2.5.3	赋值与算术的复合	19
2.5.4	自增、自减运算符	19
3	分支结构	21
3.1	关系表达式	21
3.2	逻辑表达式	21
3.3	if语句	22
3.3.1	if语句的形式	22
3.3.2	if语句的嵌套	22
3.3.3	条件表达式	22
3.4	switch语句	23
4	循环结构	24
4.1	while循环	24
4.1.1	while语句	24
4.1.2	do-while语句	24
4.2	for循环	24
4.2.1	for语句	24
4.2.2	,运算符	25
4.2.3	杂项	25
4.3	循环的中途退出	25
4.4	循环结构相关算法	26
4.4.1	枚举法/遍历法	26
4.4.2	贪婪法	26

5	数组	27
5.1	一维数组	27
5.1.1	一维数组的定义	27
5.1.2	一维数组元素的访问	27
5.2	查找	27
5.2.1	顺序查找	27
5.2.2	二分查找	28
5.3	排序	28
5.3.1	选择排序	28
5.3.2	冒泡排序	28
5.4	二维数组	29
5.4.1	二维数组定义	29
5.4.2	二维数组元素引用	30
5.5	字符串	30
5.5.1	字符数组	30
5.5.2	字符数组的输入输出	31
5.5.3	字符串处理函数库	31
5.5.4	string类	32
6	函数	33
6.1	函数定义	33
6.1.1	函数的结构	33
6.1.2	return语句	33
6.1.3	函数机制的优势	33
6.2	函数的使用	34
6.2.1	函数原型声明	34
6.2.2	函数调用	34
6.3	变量的作用域	35
6.4	变量的存储类别	36
6.4.1	自动变量	36
6.4.2	静态变量	36
6.4.3	寄存器变量	37
6.4.4	外部变量	37
6.5	数组作为函数参数	38
6.6	默认值函数	38
6.7	内联函数	38

6.8	函数重载	39
6.9	函数模板	39
6.9.1	函数模板的定义	39
6.9.2	函数模板的使用	40
6.10	递归函数	40
6.10.1	递归函数的概念	40
6.10.2	递归函数的使用	40
7	指针	42
7.1	指针概述	42
7.1.1	指针与间接访问	42
7.1.2	指针变量的定义	42
7.1.3	指针变量的基本操作	42
7.2	指针运算与数组	43
7.2.1	指针的加减运算	43
7.2.2	指针与数组元素	43
7.3	指针与函数	44
7.3.1	指针作为形式参数	44
7.3.2	数组作为函数参数再讨论	45
7.3.3	指针作为返回类型	45
7.4	动态内存分配	45
7.4.1	动态变量	45
7.4.2	动态变量的创建	45
7.4.3	动态变量的消亡	46
7.4.4	内存泄漏	46
7.5	字符串再讨论	46
7.6	引用类型	47
7.6.1	引用的定义及应用	47
7.6.2	函数参数的引用传递	47
7.6.3	返回引用的函数	48
7.7	指针数组与多级指针	48
7.7.1	指针数组	48
7.7.2	二维数组与指针	48
7.7.3	多级指针	48

8 结构体	49
8.1 结构体类型定义	49
8.2 结构体对象定义	49
8.3 结构体变量使用	49
8.4 结构体作为函数参数	50
9 模块化开发	51
10 创建新的类型	52
10.1 面向对象程序设计	52
10.1.1 抽象的过程	52
10.1.2 面向对象程序设计的特点	52
10.1.3 库与类	52
10.2 类的定义	54
10.3 对象的使用	55
10.3.1 对象的定义	55
10.3.2 对象的操作	55
10.3.3 this指针	55
10.4 对象的构造与析构	55
10.4.1 构造函数	55
10.4.2 初始化列表	56
10.4.3 析构函数	56
10.4.4 复制构造函数	57
10.5 常量与类	57
10.5.1 常量数据成员	57
10.5.2 常量对象	57
10.5.3 常量成员函数	58
10.6 静态成员	58
10.6.1 静态数据成员	58
10.6.2 静态成员函数	58
10.6.3 静态常量数据成员	59
10.7 友元	59
11 运算符重载	60
11.1 运算符重载的意义	60
11.2 运算符重载的格式	60
11.3 特殊运算符的重载	61

11.3.1 赋值运算符重载	61
11.3.2 下标运算符重载	61
11.3.3 函数调用运算符重载	61
11.3.4 自增和自减运算符重载	62
11.3.5 流运算符重载	62
11.4 自定义类型转换函数	63
11.4.1 内置类型数据转换为类的对象	63
11.4.2 类的对象转换为其他类型	63
12 组合与继承	64
12.1 组合	64
12.2 继承	64
12.2.1 派生类的定义	64
12.2.2 派生类对象的构造和析构	65
12.2.3 重定义基类的函数	65
12.2.4 派生类对象与基类对象	65
12.3 多态性	65
12.3.1 多态性	65
12.3.2 虚函数	65
12.3.3 虚析构函数	66
12.4 纯虚函数与抽象类	66
13 输入输出与文件	67
13.1 流与标准库	67
13.2 输入输出与缓冲区	67
13.3 基于控制台的输入输出	67
13.4 基于文件的输入输出	68
14 异常	70

1 绪论

1.1 程序设计概述

- 程序设计是教会计算机如何解决问题，设计出计算机完成某个任务的程序的过程
- 分为面向过程的程序设计和面向对象的程序设计

1.2 计算机组成

能够按照事先存储的程序，自动、高效地对数据进行输入、处理、存储、输出的系统，称为计算机

1.2.1 计算机硬件

冯·诺依曼结构：计算机有存储器、运算器、控制器、输入输出设备4大组成部分，其中运算器和控制器合称中央处理器 (CPU)

- 存储器：保存正在运行的指令和数据
 - 最小的存储单元是比特 (bit)，1个比特存储1个二进制位
 - 8个比特组成1个字节 (Byte)，若干字节组成1个字 (Word)
- 运算器：执行程序中的指令，由逻辑电路和寄存器组成
 - 逻辑电路：执行控制器发出的信号对应的操作
 - 寄存器：存放正在运算的数据
- 控制器：控制计算机的其他部分完成计算机的指令
 - 程序计数器：记录下条指令的地址
 - 指令寄存器：暂存正在执行的指令
 - 指令译码器：识别指令功能，分析指令要求
 - 时序控制电路：生成时序信号
 - 微操作控制电路：执行各种操作命令
- 输入输出设备：人与计算机进行交互的渠道

1.2.2 计算机软件

计算机软件：决定计算机功能，包括系统软件和应用软件

- 系统软件：将用户与计算机隔离，使用户可以通过计算机软件控制计算机硬件
- 应用软件：为了支持特定的、实际的用途而开发的软件

1.3 程序设计语言

1.3.1 机器语言

机器语言：第一代计算机语言，每条语句都是一个二进制比特串（01串）

- 可由计算机硬件直接识别并执行
- 人类难以理解，且几乎不可移植

1.3.2 汇编语言

汇编语言：用英文缩写作为助记符代替01串

- 可由汇编器直接转化为机器语言
- 解决了可读性问题，没有解决不可移植问题

1.3.3 高级语言

高级语言：更接近于人类语言（英语），表达形式更接近科学计算

- 需要编译器翻译为机器语言后才能执行
- 在一定程度上解决了可移植性问题（某些方面对于不同机器仍有歧义）

1.4 程序设计过程

1.4.1 算法设计

- 算法：使用计算机解决某一问题的方案，是程序设计的“灵魂”
- 难点：如何利用计算机的简单功能完成复杂任务
- 特点：
 - 无二义性（每一步都能精确转化为计算机的特定操作）
 - 有效性（每个步骤都可由计算机的基本操作实现）
 - 有限性（执行有限步骤后可得到结果）
- 表示：自然语言、流程图、伪代码

1.4.2 编程

- 程序：用高级语言描述的算法
- 源文件：存储在计算机中的程序
C++程序源文件扩展名为 `.cpp`

1.4.3 编译与链接

- 编译器：将高级语言转化为机器语言的程序
- 目标程序：编译器翻译高级语言程序源文件得到的机器语言程序
- 链接：将目标程序与已有程序放在一起的过程
- 可执行文件：链接以后能够运行的程序

1.4.4 调试与维护

- 调试：在程序运行的过程中设置断点，比较程序运行的中间结果与期望的中间结果，从而找出并改正逻辑错误的过程
- 维护：使用程序的过程中发现程序的漏洞并修复的过程

2 程序基本结构

2.1 程序的基本结构

C++程序结构：一个C++程序由注释、预编译指令、名字空间指令、主程序组成，其中主程序至少要包含 `main` 函数

2.1.1 注释

- 注释与程序：注释在编译过程中会被忽略，不会影响程序行为
- 注释的目的：
 - 解释程序中一些比较难以理解的部分
 - 备注信息：作者、修改历史、开源协议等
 - 临时让某些代码无效，便于调试
- 注释的格式：
 - `/*` 和 `*/` 之间的所有文字
 - `//` 到此行结束的所有文字

简单的注释示例：

```
/* File: main.cpp
 * Term: 2023-2024-1
 * Chapter: 2
 */

// Print "Hello World!" on the screen
```

2.1.2 预编译指令

- 预编译指令：在编译之前处理的命令
 - 以 `#` 开头，无需分号结尾
- 一类常用的预编译命令为库包含
 - 用 `#include` 实现，表示包含某个库
- 库：程序员编写的能够实现特定功能的子程序
 - 每个库要提供接口
 - C/C++为 `.h` 文件
 - C++还可以用 `.hpp` 文件
 - 头文件告诉用户和编译器该库提供了哪些功能，以及如何使用特定功能
 - 编译器也可以根据头文件检查程序中对对应库的调用是否正确
- 库包含的格式：

- `#include <filename>` 包含系统库，例如包含标准输入输出流对象的头文件：

```
#include <iostream>
```

- `#include "filename.h"` 包含自定义库，例如包含位于当前文件夹下用户自定义的有理数库：

```
#include "rational.h"
```

2.1.3 名字空间

- 名字空间：把一组实体组合在一起，构成作用域，用于解决重名问题
- C++标准库中的实体定义在名字空间 `std` 中
- 可用如下方式表示默认使用 `std` 中的实体：

```
using namespace std;
```

这样 `std::cout` 就可以简写为 `cout`

2.1.4 主程序

- 主程序：算法的描述
 - 每个程序都是由一个或多个函数组成
 - 其中至少要包含唯一的 `main` 函数，它是程序的入口
 - `int main()` 称为函数头，`{ }` 及其内容称为函数体

至此我们就可以写出在屏幕上打印 Hello World! 的C++程序：

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello World!" << endl;
    return 0;
}
```

2.2 变量与常量

- 变量：编写程序时未确定或程序运行过程中会变化的量
 - 提供具名的、可供程序操作的存储空间
 - 临时存储输入数据、中间结果和最终结果
- 常量：编写程序时已确定且程序运行过程中保持不变的量

2.2.1 变量定义

- 变量3要素：数据类型、名称、值
- 定义格式：<type> <variable 1>, <variable 2>, ..., <variable n>;
 - <type> 为类型名
 - <variable i> 为变量名
- 变量在使用前必须被定义
- 命名规则：
 - 首字符为字母或下划线
 - 其他字符为字母或数字或下划线
 - 不可与系统关键字相同（如 `int`, `while`, `return` 等），因为关键字在C++中有特殊用途
 - 区分大小写，大小写不同时变量不同
 - 标识符的最大长度由编译器指定
 - 标识符应尽量表明变量含义
- 定义位置：任意，但推荐在首次使用前定义，防止定义后未使用，造成空间浪费
- 变量初始化：包括变量定义、变量赋初值
 - 变量定义的目的是为变量分配存储空间
 - 变量赋初值的目的是为分配的存储空间指定内容
- 格式：
 - <type> <variable> = <initial value>;
 - <type> <variable>(<initial value>);
 - <type> 为类型名，<variable> 为变量名，<initial value> 为初值

示例：定义了 3 个变量，其中变量 b 未赋初值

```
int a=5,b;  
char ch('A');
```

若变量定义时未赋初值，则变量的值为变量的存储空间上原有的内容，可能会导致程序错误，因此给变量赋初值是良好的习惯

2.2.2 数据类型

- 分类：内置数据类型、自定义数据类型¹
- 特征：数据在内存中是如何表示的，也决定了值的范围
 - 数据占据内存空间的大小
 - 数据的布局方式

¹有关自定义数据类型的内容，详见第8～12章

- 整型变量:
 - 分类:
 - * 按占用内存:
 - 长长整型 `long long int` 【C++11新增】
 - 长整型 `long int`
 - 基本整型 `int`
 - 短整型 `short int`
 - * 按符号类型:
 - 有符号: 直接用上述关键词
 - 无符号: `unsigned`+上述关键词
 - 占用内存: C++语言规定了各类型的最少内存
 - * `long`: 4字节
 - * `short`: 2字节
 - * `int` $\geq$ `short`
 - * 具体占用字节数由编译器根据操作系统位数指定
 - 允许运算:
 - * 算术运算: `+`, `-`, `*`, `/`, `%`
 - `/`为取商的整数部分
 - `%`为取余数, 称为模
 - * 比较运算
 - * 输入输出运算
 - 机内表示: 以补码表示
 - * 原码: 最高位是符号位, 其余为数值位
 - 符号位: 0表示正数, 1表示负数
 - 数值位: 二进制表示
 - * 反码: 正数的反码与原码相同, 负数的反码为其绝对值的二进制表示取反
 - * 补码: 正数的反码与原码相同, 负数的反码为其反码+1
 - 溢出问题: **整数运算要注意范围**。如1字节的数0111 1111表示127, 加1变为1000 0000表示-128, 称为溢出。C++程序不会检查溢出, 需要编程者自己注意
- 浮点型:
 - 机内表示: 浮点表示法 $a \cdot 2^b$
 - * a 称为尾数, b 称为指数
 - * 浮点数无法在机内精确表示
 - 分类:
 - * 单精度 `float`
 - * 双精度 `double`

- * 长双精度 `long double`
- 精度: C++规定了最小精度位数:
 - * `float`: 6位十进制有效数字
 - * `double`: 10位十进制有效数字
- 允许运算:
 - * 算术运算: `+`, `-`, `*`, `/`, 但不允许 `%`
 - * 比较运算
 - * 输入输出运算
- 字符型 `char`
 - 存放1个字母或符号
 - 占1字节, 在机内用字符编码表示
 - 常用的字符编码表是ASCII
 - 特性: `0 ~ 9`, `A ~ Z`, `a ~ z`分别连续、顺序存放
 - 只有编码`0 ~ 127`有效
 - 允许运算: 算术运算、比较运算、输入输出运算
 - ASCII表中部分字符为非打印字符, 直接输出不会显示在屏幕上
- 布尔型 `bool`: 【C++新增】
 - 占1字节, 值为 `true` 或 `false`
 - 允许运算: 算术运算、比较运算、逻辑运算
 - 进行输入输出运算时不显示 `true` 或 `false`, 而是显示 `1` 或 `0`
 - 在C中使用 `bool` 类型变量要使用如下预编译指令

```
#include <stdbool.h>
```

- 求占用空间: `sizeof()` 函数
 - `sizeof(<type>)` 表示求变量类型 `<type>` 占据的字节数
 - `sizeof(<variable>)` 表示求变量 `<variable>` 占据的字节数
 - `sizeof(<expression>)` 表示求表达式 `<expression>` 的结果占据的字节数
- 变量别名: 给变量类型重新命名
 - 格式: `typedef <original name> <current name>;`
 - `<original name>` 是原名, `<current name>` 是现名
 - 之后可以用该变量类型的原名或现名定义变量

2.2.3 常量与符号常量

- 整型常量
 - 进制表示:
 - * 默认十进制, 如 `123`

- * 八进制，如 0123
 - * 十六进制，如 0x123
- 默认标准整型 `int`
 - * 如需看作长整型，可在后面加 `L`，如 123L
 - * 如需看作无符号数，可在后面加 `U`，如 123U
- 浮点数常量
 - 十进制表示，如 3.14
 - 科学计数法，如 `2e4` 表示 2×10^{-4}
 - * 尾数不能为空（1 也不能省略）
 - * 指数必须是整数
 - 一般作 `double` 处理
 - * 如果要看作 `float`，可在后面加 `F`，如 3.14F
- 字符常量
 - 可打印字符：用单引号引起来的1个字符，如 `'D'`
 - 非打印字符：用“转义序列+可打印字符”表示
 - 反斜杠符号 `\` 称为转义字符
 - 常用非打印字符：
 - * `\n` 换行符
 - * `\t` 制表符 (Tab)
 - * `\h` 退格符 (Backspace)
 - * `\r` 回车符 (Enter)
 - * `\f` 换页符
 - * `\'` 单引号（仅用于字符常量，如 `'\''` 表示字符 `'`；字符串常量中的单引号不要转义）
 - * `\"` 双引号（仅用于字符串常量，如 `"\"Hello\""` 表示字符串 `"Hello"`；字符中的双引号不要转义）
 - * `\\` 反斜杠本身
- 布尔常量：`true` 和 `false`
- 符号常量：给程序中的常量取别名
 - 宏定义：【C/C++】
 - * 格式：`#define <constant> <name>`
`<constant>` 是原名，`<name>` 是现名
 - * 位置：在预编译处
 - * 原理：编译器会将 `<constant>` 替换为 `<name>`
 - * 不会检查数据类型
 - `const` 常量：【C++】

- * 格式: `const <type> <constant name> = <constant value>;`
 <type> 是类型, <constant name> 是常量名, <constant value> 是常量值
- * 位置: 在常量使用前
- * 定义时必须赋初值, 而且只能在定义处修改该常量的值
- * <constant name> 命名要求与标识符命名要求相同
- * 通常使用全大写字母命名
- * 好处: 提高程序可读性; 在改变常量时只需修改定义处的值即可一改全改; 编译器会检查数据类型

2.3 输入与输出

2.3.1 输入

- 实现: 输入流对象 `cin` 和流提取运算符 `>>`
- 格式: `cin >> <variable 1> >> <variable 2> >> ... >> <variable n>;`
 其中 <variable i> 是变量名
- 响应: 程序执行到 `cin` 语句会暂停, 等待用户输入, 用户输入回车时继续
 - 本质上是 `cin` 与 <variable 1> 的 `>>` 运算
 - 用户输入后, 程序先判断 <variable 1> 的数据类型, 然后跳过空白字符, 最后读取变量值
 - 用户输入合法时, `cin` 与 <variable 1> 的 `>>` 运算结果为 `cin`, 由程序继续读取, 或语句结束
 - 用户输入非法时, `cin` 与 <variable 1> 的 `>>` 运算结果为某一输入错误标记, 而该输入错误标记与 <variable 1> 的 `>>` 运算将什么也不做, 即程序忽略错误处理后的语句剩余部分
- 空白字符处理: `cin.get()` 函数
 - 当用户希望字符变量 `char` 的输入值为空白字符时, `cin` 和 `>>` 无法实现, 因为会跳过空白字符
 - 实现方法: `cin.get(<char>);` 或 `<char> = cin.get();`

2.3.2 输出

- 实现: 输出流对象 `cout` 和流插入运算符 `<<`
- 格式: `cout << <param 1> << <param 2> << ... << <param n>;`
 - <param i> 可以是变量、常量、表达式, 分别打印变量值、常量值、表达式结果
 - <param i> 也可以是 `endl`, 表示换行

2.4 算术运算

2.4.1 算术表达式

- 作用：完成计算的语句
- 组成：算术运算符和操作数
- 算术运算符：+（加法）-（减法）*（乘法）/（除法，其中整型数据的除法只取商）%（模，仅用于整数运算，除法取余数）
- 运算顺序：先算括号里面的，多级括号先算最内层，同层括号先算乘除模后算加减
- 不允许用中括号[]和大括号{}，因为中括号和大括号在C++中有其他用途；多层括号只能用多层小括号嵌套
- 结合性：左结合，即同级运算从左到右
- 运算对象：整型、浮点型、字符型、布尔型（变量或常量）

2.4.2 不同类型数据的运算

- 机理：C++程序只能执行同类型数据的运算，且运算结果与运算数的数据类型相同；在执行不同类型数据的运算时要先转化为同一数据类型
- 自动转换：由编译器完成的数据类型转换
 - 字符型、布尔型转化为整型
 - 较短的整型转化为较长的整型
 - 整型转化为浮点数
 - 较粗略的浮点数转化为较精确的浮点数
- 自动转换原则：
 - 占用空间少的向占用空间多的转换
 - 数值范围小的向数值范围大的转换

2.4.3 强制类型转换

- 格式：
 - (`<type>`)(`<expression>`)
 - `<type>`(`<expression>`)
 - `<type>` 是目标类型名，`<expression>` 是待转换的表达式
- 强制类型转换可能产生错误，应尽量避免使用

2.4.4 数学函数库

- 除+, -, *, /, %之外的数学运算需要通过函数实现
 - `<cmath>` 库提供了部分函数

- 可通过预编译指令包含该库，此后就可在主程序中使用库函数：
 - * `#include <math.h>` 【C的库包含】
 - * `#include <cmath>` 【C++的库包含】
- 主要函数：（默认操作数和结果都是 `double`）
 - 绝对值 $|x|$
 - * `abs(x)`, x 为 `int` 时使用
 - * `fabs(x)`, x 为 `double` 时使用
 - 指数 e^x 为 `exp(x)`
 - 幂运算 x^y 为 `pow(x, y)`
 - 平方根 $\sqrt{x}$ 为 `sqrt(x)`
 - 对数：
 - * $\ln x$ 为 `log(x)`
 - * $\lg x$ 为 `log10(x)`
 - 三角函数 $\sin x, \cos x, \tan x$ 分别为 `sin(x), cos(x), tan(x)`
 - 反三角函数 $\arcsin x, \arccos x, \arctan x$ 分别为 `asin(x), acos(x), atan(x)`

2.5 赋值运算

2.5.1 赋值表达式

- 格式：`<variable> = <expression>;`
 - 其中 `<variable>` 为变量名，`<expression>` 为表达式
- 原理：
 1. 计算 `<expression>` 的值
 2. 将结果存入 `<variable>`
 3. 用变量的最终值替换程序中的 `<variable> = <expression>` 并继续执行
- 程序语言与代数式的区别：在代数中 $x = x + 1$ 表示一个无解的方程，而在C/C++语言中 `x=x+1` 表示取出变量 `x` 中存放的值，令该值 +1，之后将结果存入变量 `x`
- 左值与右值：
 - 左值：能出现在赋值运算符左边或右边的值
 - 右值：只能出现在赋值运算符右边的值
 - 左值必有内存空间
- 类型转换：根据变量的数据类型，强制转换表达式结果的数据类型
 - `float` 和 `double` 取整后转换为 `int`
 - `int` 数值不变转换为 `float` 和 `double`
 - `char` 取机内码转换为 `int`
 - `int` 取机内表示的最低8位转换为 `char`

- bool 转换为 int 时, 取 true=1, false=0
- int 转换为 bool 时, 为0转换为 false, 不为0转换为 true

2.5.2 多重赋值

- 赋值运算是右结合的, 即同级运算从右到左
 - 因此可在统一赋值表达式中同时给多个左值赋值
 - 优点: 方便, 代码量小
 - 缺点: 降低了可读性; 可能产生难以预料的错误

考察如下代码段:

```
double d;  
int i;  
d = i = 1.5;
```

根据代码原理, 1.5 转换为 1 后存入变量 i, 然后用 1 替换 i = 1.5, 则后面执行的是 d=1, 1 转换为 1.0 后存入变量 d, 可能与预期得到的 1.5 不符

2.5.3 赋值与算术的复合

- 为使代码更加简洁可读, 在C/C++中以下2个表达式等效:
 - <variable> = <variable> <operator> <expression>;
 - <variable> <operator>= <expression>;
 - <variable> 为变量名, 且本处出现的所有 <variable> 代表相同变量
 - <operator> 为运算符名, 且本处出现的所有 <operator> 代表相同运算符
 - <expression> 为表达式, 且本处出现的所有 <expression> 代表相同表达式
- 如 x=x+1; 可简写为 x+=1;

2.5.4 自增、自减运算符

- 格式:
 - 前缀表达式: <variable>++, <variable>--
 - 后缀表达式: ++<variable>, --<variable>
 - 其中 <variable> 为变量名, 一般为整型变量
- 原理:
 - ++ 使变量 +1, -- 使变量 -1
 - 前缀表达式的结果为变量自增或自减之前的值, 后缀表达式结果为变量自增或自减之后的值

考察如下代码段:

```
int i = 2;  
int j = ++i;  
int m = 2;  
int n = m--;
```

则：最终 i=3, j=3, m=1, n=2

3 分支结构

3.1 关系表达式

- 作用：比较两个值
- 关系运算符：
 - <（小于）
 - <=（小于等于）
 - >（大于）
 - >=（大于等于）
 - ==（等于）
 - !=（不等于）
- =表示赋值运算，==表示相等
- 关系表达式：<expression 1> <relation operator> <expression 2>
 - 其中 <expression> 为表达式，<relation operator> 为关系运算符
 - 结果为 bool 型变量（true 或 false）
- 结合方式：左结合
 - 如：-2<-1<0 $\Leftrightarrow$ (-2<-1)<0 $\Leftrightarrow$ true<0 $\Leftrightarrow$ 1<0
 - 所以 -2<-1<0 的结果为false，与纯代数运算的结果不同

3.2 逻辑表达式

- 作用：处理多个 bool 变量为1个 bool 变量
- 逻辑运算符：
 - !（一元运算符，非）
 - ||（二元运算符，或）
 - &&（二元运算符，与）
- 运算结果：
 - !a 的运算结果与 a 相反
 - a||b 运算结果为 false 当且仅当 a 和 b 均为 false，其余情况均为 true
 - a&&b 运算结果为 true 当且仅当 a 和 b 均为 true，其余情况均为 false
- 编译器对执行过程的优化：部分编译器在编译时会作出以下优化
 - <expression 1> && <expression 2> 中，若 <expression 1> 为 false，则跳过 <expression 2> 直接返回 false
 - <expression 1> || <expression 2> 中，若 <expression 1> 为 true，则跳过 <expression 2> 直接返回 true
 - 具体优化与否要根据编译器版本而定，因此不推荐在逻辑表达式中包含赋值语句

- 优先级：从上到下依次降低

- !
- *,/,%
- +,-
- <,>,<=,>=
- ==,!=
- &&
- ||

3.3 if语句

3.3.1 if语句的形式

- 格式：
 - `if (<expression>) {<codeblock 1>}`
 - * 若 `<expression>` 为 `true`, 则执行 `<codeblock 1>` 的所有语句
 - * 否则跳过 `<codeblock 1>`, 即什么也不做
 - `if (<expression>) {<codeblock 1>} else {<codeblock 2>}`
 - * 若 `<expression>` 为 `true`, 则执行 `<codeblock 1>` 的所有语句
 - * 否则执行 `<codeblock 2>` 的所有语句
- 子句: `<codeblock i>` 可以为单个语句（此时可以省略 `{}`），也可以是多个语句
 - `<expression>` 为 `true` 执行的语句块称为`then`子句
 - `<expression>` 为 `false` 执行的语句块称为`else`子句

3.3.2 if语句的嵌套

- if 语句的嵌套: `then`子句或`else`子句含有 `if` 语句的情况
- `else` 的配对: 向前寻找最近的没有与 `else` 配对的 `if`, 但不会到 `{}` 内部与 `if` 配对
- 用缩进关系表示`if`与`else`的配对情况是良好的编程习惯

3.3.3 条件表达式

- 条件运算符: `?:` (三元运算符同时出现, 一起使用)
- 条件表达式: `<expression> ? <value 1> : <value 2>`
 - 若 `<expression>` 为 `true`, 则结果为 `<value 1>`
 - 若 `<expression>` 为 `false`, 则结果为 `<value 2>`

3.4 switch语句

- 格式:

```
switch(<expression>)  
{  
    case <constant 1> :  
        <codeblock 1>  
    case <constant 2> :  
        <codeblock 2>  
    /* Other case statement */  
    case <constant n> :  
        <codeblock n>  
    default :  
        <codeblock n+1>  
}
```

<expression> 是表达式, <constant i> 是常量, <codeblock i> 是语句块

- 执行:
 1. 计算 <expression> 的值
 2. 将结果从 <constant 1> 开始匹配
 3. 若匹配到了 <constant i>, 则不再匹配, 从 <codeblock i> 开始执行
 4. 若未匹配到任何常量, 则匹配结束, 从 <codeblock n+1> 开始执行
- default : <codeblock n+1>也可直接省略, 此时若未匹配到任何常量, 则会直接退出 switch 语句块
- break 关键字: 在 switch 语句中, 若执行到 break; 语句, 则会直接跳出 switch 语句; 适用于在执行到下一个 case 或 default 前取消执行后面的语句块的情况
- <expression> 的要求: 应为整型表达式 (包括结果能转化为整型的表达式)

4 循环结构

4.1 while循环

4.1.1 while语句

- 格式: `while (<expression>) {<codeblock>}`
 - `<codeblock>` 称为循环体
 - 若循环体只有1条语句, 可省略 `{}`
- 原理: 计算 `<expression>` 的值
 - 若为 `false`, 则退出循环体
 - 若为 `true`, 则执行 `<codeblock>` 中的语句, 之后回到第1步

4.1.2 do-while语句

- 作用: 确保循环体至少被执行了1次
- 格式: `do {<codeblock>} while (<expression>)`
若循环体只有1条语句, 可省略 `{}`
- 原理: 先执行1次循环体, 再按照普通 `while` 语句的方式执行

4.2 for循环

4.2.1 for语句

- 格式: `for (<expression 1>; <expression 2>; <expression 3>) {<codeblock>}`
 - `for (<expression 1>; <expression 2>; <expression 3>)` 称为循环控制行
 - 若循环体只有1条语句, 可省略 `{}`
- 原理:
 1. 执行 `<expression 1>`
 2. 计算 `<expression 2>` 的值
 - 若为 `false`, 则退出 `for` 循环
 - 若为 `true`, 则执行 `<codeblock>` 中的语句, 之后跳到第3步
 3. 执行 `<expression 3>`, 之后回到第2步
- 作用: 便于设置循环次数
 - `<expression 1>` 用于初始化循环变量
 - `<expression 2>` 用于设置循环条件
 - `<expression 3>` 用于对循环变量进行操作
 - 常用 `i,j,k` 作为循环变量名, 例如按定义实现阶乘:


```
// Suppose that n is known
int result = 1;
for (int i = 1; i <= n; i++)
    result *= i;
```

- 循环变量与销毁：定义在循环控制行内的变量将在循环结束后销毁，不再可用
 - 例如上面阶乘中，for 循环退出之后变量 i 被销毁，之后使用 i 会报错
 - 定义在循环控制行外的变量不受影响，循环结束后仍可使用，并且保留循环过程中的变化，例如上面阶乘中的 n

4.2.2 ,运算符

- 格式：<expression 1>, <expression 2>, ..., <expression n>;
- 执行：从左到右依次计算各个表达式的值
 - 整个语句的返回值为 <expression n> 的值
 - ,运算符是所有运算符中优先级最低的
- 作用：整合多个表达式为单个语句

4.2.3 杂项

- 表达式省略：for 循环控制行中3个表达式都可以省略
 - 默认空表达式结果为 true
 - ; 不可以省略
- 死循环：若 <expression 2> 的结果一直为 true，且没有中途退出机制，则循环体不能退出
 - 最简单的死循环为 for (; ;) ;，即循环控制行和循环体都为空
 - 可通过设置循环条件或中途退出机制避免死循环
- 循环嵌套：允许 for 循环和 while 循环互相嵌套
 - 在外层循环的每个循环周期，相邻内层循环执行所有循环周期
 - 注意循环变量在内外层循环间的作用，如果不希望互相干扰，要设置不同的循环变量

4.3 循环的中途退出

- 退出全部循环周期：循环体中读到 break; 语句会结束循环
- 退出单个循环周期：循环体中读到 continue; 语句会直接跳过本周期的剩余语句，直接进入计算循环条件并判断的环节
- 在循环嵌套中只会退出单层循环的周期

4.4 循环结构相关算法

4.4.1 枚举法/遍历法

- 定义：列举所有可能是解的方案，并进行检验和筛选
- 优点：简单易理解；便于保证正确性
- 缺点：处理问题的时间效率差
- 优化：减少解的选择

4.4.2 贪婪法

- 定义：寻求局部最优，之后组合局部最优成为全局最优
- 优点：能快速缩小问题的规模
- 缺点：使用情况有限
 - 局部最优的组合可能不符合问题条件
 - 局部最优的组合可能不是全局最优

5 数组

5.1 一维数组

5.1.1 一维数组的定义

- 格式: `<type> <name>[<size>];`
 - `<type>` 为数据类型
 - `<name>` 为数组名, 遵循标识符命名规则
 - `<size>` 为数组长度, 应尽量为常量
- 初始化:
 - 全部初始化, 如 `int a[5] = {0, 1, 2, 3, 4};`
 - 部分初始化, 如 `int a[10] = {0, 1, 2, 3, 4};`
此时 `a[]` 中后5个元素为0
 - 通过初始化确定数组规模, 如 `int a[] = {0, 1, 2, 3, 4};`
此时 `a[]` 的长度自动为5

5.1.2 一维数组元素的访问

- 格式: `<name>[<index>]`
 - `<name>` 为数组名
 - `<index>` 为下标, `[]` 称为下标运算符
 - 当数组的长度为 n 时, 下标的范围为 $0 \sim n - 1$
 - 下标可为整数、整型变量、结果为整型的表达式
- 下标越界: 数组长度为 n 时出现 $index < 0$ 或 $index > n - 1$ 的情况
此时访问的是数组范围以外的地址上的元素, 会得到随机值影响程序的运行, 严重时还可能导致程序崩溃

5.2 查找

查找操作: 检查特定元素在特定数组中是否存在, 若存在则输出下标位置, 否则输出表示查找失败的特定标记

5.2.1 顺序查找

- 定义: 从数组的首个元素开始逐渐向后比较, 直到找到目标元素或者数组结束, 返回下标或失败标记
- 实现: 利用循环遍历数组元素

5.2.2 二分查找

- 定义：在数组已经按照某一顺序排序的前提下，检查中间元素，每次将查找范围缩小一半，可提高查找速度
- 实现：假设要查找的元素为 `key`，数组为 `a[n]`
 1. 初始化查找边界，如 `left=0, right=n-1`;
 2. 计算中间元素的位置，如 `mid=(left+right)/2`;
优化：为防止 `left+right` 溢出，可用 `mid=left+(right-left)/2` 代替
 3. 比较 `key` 与 `a[mid]`
 - 若 `key==a[mid]`，则返回 `mid`，查找结束
 - 若 `key<a[mid]`，则修正查找范围为前半，即 `right=mid-1`;
 - 若 `key>a[mid]`，则修正查找范围为后半，即 `left=mid+1`;
 4. 检查此时边界是否合理
 - 若 `left>right`，则要查找的元素不存在，返回错误标记
 - 若 `left<=right`，则返回第2步继续查找

5.3 排序

5.3.1 选择排序

- 定义：每次选择目标元素（如最小元素）放在其最终位置
- 实现（伪代码）：以从小到大排序为例

```
for (int i=0; i<n; i++) // The first i element(s) is/are in order
{
    search a[k], which is minimum in a[i..n-1]
    swap a[k], a[i]
}
```

需要用 `for` 循环的嵌套实现

- 注意：交换需要额外分配空间暂存数据，否则会导致数据丢失

例如下面的写法是错误的：`a[k]=a[i]; a[i]=a[k];`

因为若 $a[k] = x, a[i] = y$ ，则程序执行 `a[k]=a[i];` 后 $a[k] = a[i] = y$ ，会导致 x 丢失

正确的写法是：`int tmp=a[k]; a[k]=a[i]; a[i]=tmp;`

5.3.2 冒泡排序

- 定义：通过不断调整不符合次序的元素的位置实现排序
- 每次扫描依次比较相邻元素，并通过交换使得该相邻元素有序
- 每轮扫描必定在最右侧产生至少1个排序好的数

- 实现（伪代码）：以从小到大排序为例

```
for (int i=0; i<n; i++) // The last i element(s) is/are in order
{
    for (int j=0; j<n-i-1; j++)
    {
        if (a[j]>a[j+1]) swap a[j],a[j+1]
    }
}
```

- 优化：如果一轮扫描过程中未发生元素交换，说明数组已排序好，可提前返回，伪代码如下

```
for (int i=0; i<n; i++) // The last i element(s) is/are in order
{
    Swapping has taken place = false
    for (int j=0; j<n-i-1; j++)
    {
        if (a[j]>a[j+1])
        {
            swap a[j],a[j+1]
            Swapping has taken place = true
        }
    }
    if (Swapping has taken place == false) break
}
```

在数组部分有序的前提下，提前返回能大幅提高代码时间效率

5.4 二维数组

5.4.1 二维数组定义

- 多维数组：以数组作为元素的数组
其中二维数组可看作是矩阵，是最常用、最基本的多维数组
- 二维数组的声明：<name> [<rows>][<columns>]
 - <name> 是数组名
 - <rows> 是行数
 - <columns> 是列数
- 二维数组的初始化：

- 全部初始化, 如 `int a[2][3]={0,1,2,3,4,5};`
用大括号包含每一行可使表示更清晰, 如 `int a[2][3]={0,1},{2,3},{4,5};`
- 部分初始化, 如 `int a[4][4]={0,1,2};`
此时 `a[i][j]` 第1行第4个元素和后3行元素均为0
- 逐行部分初始化, 如 `int a[3][3]={0,1},{2},{3}}`
此时 `a[i][j]` 为 `{{0,1,0},{2,0,0},{3,0,0}}`
- 二维数组一般不采用通过初始化确定数组长度的方法
如果需要使用, 只能通过初始化确定行数, 而列数必须提前指定

5.4.2 二维数组元素引用

- 格式: `<name> [<index1>][<index2>]`
 - 行数和列数都是从0开始的
 - 表示引用二维数组的第 $\langle index1 \rangle \times \langle columns \rangle + \langle index2 \rangle$ 个元素
 - 与一维数组不同, `<index2>` 超出数组的列数时不一定会造成越界, 只有当上述表达式的值超过二维数组的范围时才会造成越界
 - 例如在数组 `a[4][5]` 中, `a[1][5]` 和 `a[2][0]` 引用的都是第2行第0列的元素, 且前者并不会造成数组越界

5.5 字符串

字符串: 由一系列顺序结构的字符型变量组成的数据类型

5.5.1 字符数组

- 格式: `char <name>[<size>];`
 - 此种形式的字符串固定以 `'\0'` 结尾
 - 使用时要注意最多放入的字符数是数组规模-1
- 初始化:
 - 一般初始化, 如 `char ch[]={'H','e','l','l','o','\0'};`
 - 自动初始化, 如 `char ch[]="Hello";`, 此时编译器会在数组尾部自动加入 `'\0'`
- 空串: `""` 就是空串, 空串不包含任何字符, 但它的长度为1, 因为包含 `'\0'`
- 注意事项:
 - 由于读取字符串的机制是读到 `'\0'` 结束, 因此字符串不以 `'\0'` 结尾时十分危险
 - 单引号表示字符常量, 例如 `'a'` 占1字节, 而 `'aa'` 非法
 - 双引号表示字符串常量, 例如 `"a"` 占2字节, `"aa"` 占3字节, 因为以 `'\0'` 结尾

5.5.2 字符数组的输入输出

- 逐个字符的输入，例如 `cin>>ch[2]`；表示在输入缓冲区，忽略空白字符后，读取1个字符放入 `ch[2]`
也可使用 `cin.get()` 函数以读取空白字符，这与字符型变量的输入是一样的
- 整个字符串的输入，例如 `cin>>ch`；表示在输入缓冲区，忽略空白字符后，读取下个空白字符前的字符放入 `ch`
- 直接使用 `cin` 存在以下问题：
 - 无法接受空白字符
 - 无法保证输入不会造成越界
- `get` 函数：
 - 格式：`cin.get(<name>,<length>,<token>);`
 - 执行：在输入缓冲区接受字符串，直到读取的字符数达到 `<length>-1` 或遇到结束标记 `<token>`，之后放入 `<name>` 表示的字符串中
 - 不指定 `<length>` 时默认为1，这与字符型变量的输入是一样的
 - 不指定 `<token>` 时默认为换行符 `'\n'`
 - 输入结束后，结束标记仍然会留在输入缓冲区
- `getline` 函数：其他用法与 `get` 函数相同，只是会从输入缓冲区中删除结束标记

5.5.3 字符串处理函数库

标准库文件为 `string.h`，使用库函数前需要使用预编译命令 `#include <cstring>`
主要包含的库函数²如下：（`src,dst,s,s1,s2` 均为字符串，`n` 为整型数据，`ch` 为字符型数据）

- `strcpy(dst,src)` 将字符串从 `src` 复制到 `dst`，返回 `dst` 的地址
- `strncpy(dst,src,n)` 将字符串从 `src` 复制到 `dst`，直到 `src` 结束或者复制的字符数达到 `n`，返回 `dst` 的地址
- `strcat(dst,src)` 将 `src` 拼接到 `dst` 后，返回 `dst` 的地址
- `strncat(dst,src,n)` 将 `src` 拼接到 `dst` 后，直到 `src` 结束或者拼接的字符数达到 `n`，返回 `dst` 的地址
- `strcmp(s1,s2)` 按字典序比较 `s1` 和 `s2`
 - 若 `s1>s2`，返回1
 - 若 `s1==s2`，返回0
 - 若 `s1<s2`，返回-1
- `strncmp(s1,s2,n)` 按字典序比较 `s1` 和 `s2`，但至多比较 `n` 个字符
- `strchr(s,ch)` 返回 `s` 中第1次出现 `ch` 的下标
- `strrchr(s,ch)` 返回 `s` 中最后1次出现 `ch` 的下标

²有关函数的更多内容，详见第6章

- `strstr(s1,s2)` 返回 `s1` 中第1次出现 `s2` 的下标
- `strlen(s)` 返回 `s` 的长度，即字符串中的字符个数（不包括 `'\0'`）

补充说明：

- 使用 `strcpy` 和 `strcat` 时要注意内存溢出问题
- 字典序的比较过程为：从左到右依次比较字符串相同位置上字符的内码，若全部字符均相同则比较结果为相等，否则必然会遇到不同字符，此时将第1对不同字符的比较结果作为字符串的比较结果；此外，`'\0'` 的内码是0，小于任何字符，如 `"aa"` 小于 `"aaa"`，就是因为下标为2的位置上 `'\0'` 小于 `'a'`

5.5.4 string类

- C++专门用于处理字符串的标准库，提供了 `string` 类³
- 使用前需要使用预编译命令 `#include <string>`
- 相比 `char[]` 主要新增功能：
 - 自动调整长度，不需要在定义时声明数组长度
 - 允许直接进行赋值操作，如 `str1=str2;`
 - 允许用 `+` 运算符进行拼接操作，如 `str=str1+str2;`
 - 允许直接进行比较操作，如 `str1!=str2;`
 - 成员函数 `getline` 无需指定最大长度，使用格式为 `getline(cin,<string>);`

³有关类的更多内容，详见第10章

6 函数

- 定义：将实现特定功能的语句封装起来作为程序的一部分，并命名，需要该功能时，只需要给出名字和输入即可得到输出，即为**函数机制**
 - 该语句组合的命名称为**函数名**
 - 需要给出的输入称为**参数**
 - 使用该功能的过程称为**调用函数**
 - 得到的输出称为**返回值**
- 分类：
 - 库函数：系统库的函数，使用前需在预编译中包含系统库
 - 自定义函数，封装后也可成为自定义库⁴

6.1 函数定义

6.1.1 函数的结构

- 格式：`<type> <name>(<param>) {<codeblock>}`
 - `<type>` 是**返回类型**；如果函数没有返回值，此处用 `void`，此时函数可称为过程
 - `<name>` 是**函数名**，要符合标识符命名规则，函数的命名表示函数的功能是良好的编程习惯
 - `<param>` 是**形式参数**
 - `<type> <name>(<param>)` 合称**函数头**
 - `<codeblock>` 是语句块，称为**函数体**，此处大括号`{}`不能省略，与函数体的语句数量无关，即使函数体为空也不能省略

6.1.2 return语句

- 如果函数的返回类型不是 `void`，则必须要返回某个值
- 格式：`return <expression>;`，其中 `<expression>` 是表达式，其数据类型与函数返回类型相同
- 执行：函数执行过程中遇到 `return` 语句时就会结束执行，将表达式的值作为返回值
- 返回类型为 `void` 的函数中也可使用 `return;` 语句直接结束函数执行

6.1.3 函数机制的优势

- 信息隐藏与代码封装：使用者无需了解函数的细节，只需知道使用方式并给出参数
- 减少工作与代码复用：同样的功能只需要实现1次，便于模块化编程

⁴有关自定义库的更多内容，详见第9章

6.2 函数的使用

6.2.1 函数原型声明

- 作用：函数在使用前必须被声明，以便编译器检查函数是否正确使用（给出的参数是否正确），以及函数返回值作为调用函数的语句的一部分是否正确使用
- 格式：<type> <name>(<param>);，类似于函数头
 - <type> 为返回类型
 - <name> 为函数名
 - <param> 为参数列表
 - * 按照 <type> <name> 的形式给出单个参数
 - * 参数与参数之间用逗号，分隔
 - 例如返回较大数的函数，其原型可声明为 `int getmax(int a, int b);`
- 函数声明、函数实现、函数调用中的形式参数顺序必须一致
- 如果函数在使用前就已被实现，则函数声明可以省略
- 函数声明的位置在函数使用之前，一般位于 `main` 函数之上或预编译命令的头文件中

6.2.2 函数调用

- 格式：<name>(<param>)
 - <name> 是函数名
 - <param> 是实际参数，要与形式参数表一一对应（即实际参数和形式参数的个数必须相同，且数据类型的排列必须相同）
 - 小括号()不能省略，与参数的类型和数量无关，即使函数没有参数也不能省略
 - 形式参数初始化的过程分为值传递和引用传递⁵
 - 值传递的过程即形式参数的初始化过程，即声明形式参数，并将实际参数的值复制给形式参数，因此实际参数的可为常量、变量、表达式、其他函数的调用等
 - 应避免写出与实际参数计算次序有关的调用，如 `f(++x, x)`，设 $x = 1$ ，则由于编译器对程序语言的解释不同，可能会产生 `f(1, 2)` 和 `f(2, 1)` 的解释，且难以提前预知
- 函数调用在被调用处的成分：
 - 作为语句，例如函数

```
void print()  
{  
    cout << "Hello World!" << endl;  
}
```

⁵有关引用传递的更多内容，详见7.6节

在调用时可直接写为

```
print();
```

– 作为操作数，如阶乘函数

```
int fact(int n)
{
    int result=1;
    for (int i=0; i<=n; i++) result *= i;
    return result;
}
```

在计算 $6! + 7!$ 时可写为

```
int x = fact(6)+fact(7);
```

– 函数调用也可作为其他函数的参数，如取较大值函数

```
int getMax(int a, int b)
{
    return (a>b) ? a : b;
}
```

在求 w, x, y, z 的最大值时可嵌套调用为

```
int largest = getMax(getMax(w,x),getMax(y,z));
```

– 一般而言，无返回值函数一般作为语句，有返回值函数一般作为操作数

- 函数调用的执行过程：

1. 在被调用处中计算实际参数的值
2. 复制实际参数的值到函数中，并以此初始化形式参数
3. 执行函数体内的语句，直到遇到 `return` 语句或函数体结束
4. 计算 `return` 的表达式值
5. 返回被调用处，用返回值替换函数调用，并继续被调用处的程序

6.3 变量的作用域

- 作用域：变量能够被存取的部分，与语句块（大括号{}及其内容）有关
- 局部变量：在语句块中定义的变量（包括 `main` 函数中定义的变量），只能在当前语句块且在定义后才能使用
- 全局变量：在所有函数之外定义的变量，可在定义处到文件结束后使用
 - 当前源代码文件的函数若要使用其他源代码文件定义的全局变量，则要用 `extern` 说明，如 `extern int a;` 表示使用其他文件里的全局变量 `a`

- 全局变量会破坏函数的独立性，一般不建议使用
- 函数不能使用被调处语句块中的变量，如自定义函数不能使用 `main` 函数中定义的变量
- 函数的形式参数以及函数中定义的变量只能在当前函数的函数体中使用，可视为当前语句块中的局部变量
- 允许局部变量与全局变量同名，此时在局部变量的作用域中，在局部变量定义之后，使用该标识符默认为局部变量（即全局变量被屏蔽）
要使用全局变量可使用域运算符 `::`，如 `::x`

6.4 变量的存储类别

- 数据的属性：包括数据的类型和数据的存储类别
 - 数据的存储类别表示变量的存储位置和生命周期
 - 因此标准的变量定义格式为 `<storage> <type> <name>;`
 - * `<storage>` 是存储类别
 - * `<type>` 是数据类型
 - * `<name>` 是变量名

6.4.1 自动变量

- 格式：去掉数据的存储类别，即 `<type> <name>;`
- 存储映像：位于函数的运行空间中，函数运行结束后该空间被释放，同时变量的空间也被释放，变量不再可用
再次进入函数时，操作系统重新分配空间，但前后分配的空间不一定相同

6.4.2 静态变量

- 格式： `static <type> <name>;`
 - `static` 是静态变量关键字
 - `<type>` 是数据类型
 - `<name>` 是变量名
- 静态全局变量：在所有函数体之外定义的静态变量，从定义处开始分配内存，程序运行结束后释放内存
与自动全局变量不同的是，只有本文件的函数能使用该变量，其他源文件中的函数不能使用该变量，也不能用 `extern` 声明该外部变量
- 静态局部变量：在函数体中定义的变量，存储空间在全局变量区
编译时为静态局部变量分配空间并初始化，函数执行结束后该变量不会消亡，调用该

函数时，程序会忽略该变量的定义部分，使用该静态局部变量时会直接使用上次函数返回时该静态局部变量的值，例如

```
int f()
{
    static int a=1;
    return a++;
}
```

则第1次调用 `f()` 返回1，第2次调用 `f()` 返回2，依此类推

- 静态变量使用规则：
 - 未初始化的静态局部变量，系统用0将其初始化
 - 静态局部变量的分配空间和初始化是编译时完成的
 - 静态局部变量在函数结束后仍存在，但函数不能访问其他函数中定义的局部变量
 - 程序运行结束后，自动局部变量、静态局部变量、静态全局变量、自动全局变量依次消亡（即释放变量的空间）
 - `static` 也可用于声明函数，此时该函数只能在当前源文件中使用

6.4.3 寄存器变量

- 格式: `register <type> <name>;`
 - `register` 是寄存器关键字
 - `<type>` 是数据类型
 - `<name>` 是变量名
- 存储映像：固定存储在寄存器中，寄存器是计算机存储结构中最接近逻辑电路的部分，因此可为频繁访问的变量加快访问速度
- 优化：较新版本的编译器可自动识别频繁使用的变量，并优化为寄存器变量
- 限制：只有自动局部变量才能定义为寄存器变量，而全局变量和静态变量都不能存储在寄存器中

6.4.4 外部变量

- 外部变量不是独立的存储类别，而是引用其他位置的全局变量
- 格式: `extern <type> <name>;`
 - `extern` 是外部变量关键字
 - `<type>` 是数据类型
 - `<name>` 是变量名
- 使用：
 - 用于函数体中，以使用在本函数的实现位置后定义的全局变量

- 用于程序源文件任意位置，以使用在其他源文件中定义的全局变量

6.5 数组作为函数参数

- 数组含义再讨论：数组实质上是连续空间的起始地址，数组元素在该连续空间中顺序排列
- 数组传参的实质：将数组的起始地址传递给函数，因此实际参数和形式参数实际上位于同一空间，在函数中更改数组元素将会对调用处程序中的数组元素生效
- 一维数组传参：为防止越界，数组的规模可作为独立参数传递，例如函数原型

```
void f(int a[], int size);
```

- 二维数组传参：第1个下标不指定，第2个下标指定，因为第2个下标表示了数组的长度，例如函数原型

```
void f(int a[][5], int lines);
```

表示向函数传递数组 `a[]`，且 `a[]` 的每个元素是长度为5的整型变量数组

6.6 默认值函数

- 定义或声明函数时，允许形式参数拥有默认值，当该形式参数未得到实际参数初始化时，形式参数使用默认值初始化
- 规则：没有默认值的参数在左，有默认值的参数在右，且必须分别连续，例如

```
void f(int a, int b=1, int c=2);
```

- 调用 `f(0)` 相当于调用 `f(0,1,2)`
- 调用 `f(0,0)` 相当于调用 `f(0,0,2)`
- 调用 `f(0,0,0)` 相当于调用 `f(0,0,0)`

不允许参数列表中有默认值的参数后面存在无默认值的参数，例如

```
void f(int a=0, int b, int c=2);
```

试图编译该语句时，编译器会报错

6.7 内联函数

- 格式：`inline <type> <name>(<param>);`
 - `inline` 是内联函数关键字
 - `<type>` 是数据类型
 - `<name>` 是函数名
 - `<param>` 是参数列表

- 区别：编译时，会直接将内联函数的机器码复制到调用处，避免函数调用的开销，可提高执行效率，但目标代码会变长
- 限制：内联函数不能递归⁶，且必须在被调用之前定义
- 不宜使用的场合：
 - 函数体代码较长，会造成编译后的目标机器码占用内存过大
 - 函数体内有长循环，此时执行函数体代码时间远大于函数调用的开销，使用内联函数意义不大

6.8 函数重载

- 重载：允许参数列表不完全相同的多个函数拥有相同的函数名，以提高函数的泛用性，例如取最大值函数的原型

```
int max(int x1, int x2);  
int max(int x1, int x2, int x3);  
int max(int x1, int x2, int x3, int x4);  
int max(int x1, int x2, int x3, int x4, int x5);  
int max(int x1, int x2, int x3, int x4, int x5, int x6);
```

完成函数的实现后，可用 `max` 函数得到2,3,4,5,6个整型变量的最大值

6.9 函数模板

- 泛型：当函数只是参数类型不同，而运行机制相同时，可将一系列函数重写为函数模板，称为C++的泛型机制
- 模板参数：将数据类型作为参数，称为模板参数
- 函数模板的调用：根据实际参数的类型取代模板参数

6.9.1 函数模板的定义

- 格式：`template < <paramtype> > <type> <name>(<param>);`
 - `template` 是函数模板关键字
 - `<paramtype>` 是模板参数列表（要用尖括号括住）
 - `<type>` 是返回类型
 - `<name>` 是函数名
 - `<param>` 是参数列表

例如取任意数据类型的较大值函数模板可写为

⁶有关递归函数的更多内容，详见6.10节

```
template <class T>
T max(T a, T b)
{
    return (a>b) ? a : b;
}
```

6.9.2 函数模板的使用

- 上述 max 函数可调用为：
 - max(2,5), 此时程序将 T 替换为 int
 - max('a','f'), 此时程序将 T 替换为 char
 - max(3.3,4.7), 此时程序将 T 替换为 float 或 double

6.10 递归函数

6.10.1 递归函数的概念

- 递归思想：将规模较大的问题转化为规模较小、结构相似的问题
- 递归函数：函数体中可能调用自身的函数
- 递归优势：降低程序编写复杂度，令程序更美观易读
- 递归代价：多次递归的函数开销非常大，很容易造成内存溢出
- 递归的注意事项：
 - 必须有递归终止条件
 - 必须有递归终止条件的相关参数
 - 该参数必须在合理的、有限的递归次数内向终止条件收敛

6.10.2 递归函数的使用

- 斐波那契数列： $f(0) = 0, f(1) = 1, f(n) = f(n-1) + f(n-2) (n \geq 2)$

```
int fibonacci(int n)
{
    if (n==0) return 0;
    if (n==1) return 1;
    return fibonacci(n-1)+fibonacci(n-2);
}
```

- 汉诺塔问题：有3根柱子，初始时第1根柱子套有自上而下直径依次增大的 n 个圆盘，要将第1根柱子上的圆盘全部移动到第2根柱子上，移动过程中要求：
 - 每次只能移动1个圆盘

- 圆盘在移动前后必须全部套在3根柱子上，不能置于其他位置
- 任意移动完成后，所有上下相邻的2个圆盘中，直径较大的圆盘必定位于直径较小的圆盘下方

由于每次只能移动1个圆盘，因此每次移动可表示为 $src \rightarrow dst$ ，其中 src 为该圆盘移动前的柱子， dst 为该圆盘移动后的柱子，求移动过程的详细步骤

思路：只有1个圆盘时，将其从初始柱子移动到目标柱子即可

多于1个圆盘时，先不移动最下方圆盘，将其上方所有圆盘从初始柱子移动到中间柱子，再将最下方圆盘从初始柱子移动到目标柱子，最后将其他圆盘从中间柱子移动到目标柱子（此过程可借助初始柱子）

该思路符合递归的条件，因此可用递归函数实现

```
void hanoi(int n, int src, int dst, int mid)
{
    if (n==1) cout << src << "->" << dst << endl;
    else
    {
        hanoi(n-1, src, mid, dst);
        cout << src << "->" << dst << endl;
        hanoi(n-1, mid, dst, src);
    }
}
```

7 指针

7.1 指针概述

7.1.1 指针与间接访问

- 直接访问：通过变量名访问某个内存地址中的数据
- 间接访问：通过指针变量储存内存地址，然后通过指针变量指向的地址访问内存中的数据
- 指针的特点：动态，灵活，危险

7.1.2 指针变量的定义

- 格式：<type> *<name>;
 - <type> 是数据类型，表示指针变量指向的地址中保存的数据类型，称为指针的基类型
 - * 是指针符号，表示后面的变量是指针
 - <name> 是指针变量名
- 连续定义时，每个变量之前都要加 *，例如
 - `int *p1,*p2;` 定义了2个指向整型数据的指针变量
 - `int *p1,p2;` 定义了1个指向整型数据的指针变量和整型变量

7.1.3 指针变量的基本操作

- 赋值：
 - 将某一变量的地址赋给指针变量，例如

```
int x,*p;  
p = &x;
```

定义了整型变量 `x`，定义了指向整型数据的指针变量 `p`，`&` 是取地址运算符，`&x` 表示取变量 `x` 的地址，`p=&x;` 表示将该地址赋给指针变量 `p`
指针变量也可在定义时就初始化，写为

```
int x,*p=&x;
```

- 同类型的指针变量之间相互赋值，例如

```
int *p1,*p2,x;  
p1 = &x; p2 = p1;
```

- 将指针赋值为空指针，不指向任何地址，使用时会报错
空指针用 `nullptr` 或 `NULL` 表示，可用于给指针变量赋值
在转换为 `bool` 型变量时，空指针转换为 `false`，其余指针变量转换为 `true`
- 定义后未赋值的指针称为野指针，野指针指向的地址随机
使用野指针可能导致内存问题，也可能影响其他变量的使用，十分危险
暂时不使用的指针一定要赋值为空指针

- 访问：

- 直接访问指针变量时，会得到指针变量的地址值，例如

```
int x, *p=&x;
cout << p << endl;
```

控制台会显示变量 `x` 的地址值

- 要访问指针变量指向的地址中保存的数据，应使用解引用运算符 `*`，例如

```
int x=4, *p=&x;
cout << *p << endl;
```

控制台会显示变量 `x` 的值，即 4

显然，对指针变量指向的地址上的内容进行操作时，也会影响其他变量，例如

```
int x=4, *p=&x;
*p = *p + 4;
```

此时输出 `x` 和 `*p` 时控制台均显示 8

7.2 指针运算与数组

7.2.1 指针的加减运算

- 指针加减 1 的操作，实质上是指针的值加减 1 个指针基类型的长度
例如，假设指针 `p` 是整型指针且初始指向地址 `0x1000`，程序中整型指针占据 4 字节，则执行 `p++` 后指针 `p` 指向地址 `0x1004`，执行 `p--` 后指针 `p` 指向地址 `0xffc`

7.2.2 指针与数组元素

- 6.5节提到，数组名的实质是连续空间的起始地址，这与指针变量的含义是相同的，因此常常可以互换使用，例如定义了

```
int *p, a[5]={0,1,2,3,4};
```

- 可以进行操作 `p=a;`，此时 `p` 指向 `a[0]`
- 可以使用 `p[3]` 访问数组元素 3
- 可以使用 `*(a+3)` 访问数组元素 3

- 指针和数组的区别：内存分配问题
 - 定义数组时，系统为数组名分配了存放起始地址的内存空间，也分配了数组各个元素的内存空间
 - 定义指针时，系统只为指针分配了存放地址的内存空间
 - 指针只有得到基类元素的地址后才能使用

7.3 指针与函数

7.3.1 指针作为形式参数

- 函数传参的实质：取实际变量对应地址的内容，将其复制到函数运行的空间中
- 指针传参的特殊性：向函数运行的空间传递地址，而该地址依然指向原变量
- 用途：修改被调用处程序中的变量值，例如考查以下函数的差别：

```
void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

执行上述函数的过程中，系统从调用处的程序，取得实际参数的值，复制到函数执行的空间，并初始化形式参数，执行结束后消亡变量 `tmp,a,b`，因此调用处的程序中实际参数的值并未改变

```
void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

执行上述函数的过程中，系统从调用处的程序，取得实际参数的地址，复制到函数执行的空间，初始化形式参数中的指针变量，但过程中对 `*a,*b` 的操作都是修改原地址上的内容，因此能修改调用处程序的变量值

- 优点：可以让函数和调用处的程序共享空间，从而操作调用处程序的变量
- 缺点：有时会导致错误或造成危险
- 其他用途：正常情况下函数只有 1 个返回值，而使用指针则允许函数通过指针变量，将想要在调用处程序使用的多个值写回调用处程序的变量中，实现类似于有多个返回值的效果

7.3.2 数组作为函数参数再讨论

- 形式参数和实际参数中数组名和指针都可以混用，例如下面的函数原型

```
void f(int *p);  
void f(int a[]);
```

在调用时的用法相同，且均可接受整型变量的数组名和指针作为实际参数

7.3.3 指针作为返回类型

- 函数原型：<type>* <name>(<param>);
 - <type> 是返回指针的基类型
 - <name> 是函数名
 - <param> 是参数列表
- 此时函数返回存放了指针基类型数据的地址

7.4 动态内存分配

7.4.1 动态变量

- 动态变量：程序运行过程中，根据程序的需求，动态产生和消亡的变量
- 动态变量的使用流程：
 1. 定义指针变量
 2. 申请动态空间并用指针指向，使用时也用指针变量访问
 3. 显式回收动态空间（程序不会自动消亡动态变量）

7.4.2 动态变量的创建

- 创建单个动态变量：new <type>
创建动态变量数组：new <type>[<size>]
创建动态变量并赋初值：new <type>(<init value>)
 - new 是申请空间关键字
 - <type> 是基类型
 - <size> 是数组长度
 - <init value> 是变量初值
- 举例：
 - int *p; *p = new int;
 - int *p; *p = new int[10];
 - int *p; *p = new int(20);

7.4.3 动态变量的消亡

- 动态变量在函数返回或程序结束后不会自动消亡，即为动态变量分配的空间仍然被程序占用，需要程序编写者显式释放空间
- 格式：
 - 消亡单个动态变量 `delete <pointer>;`
 - 消亡动态变量数组 `delete []<pointer>;`
 - `<pointer>` 是指针变量
- 实质：释放指针指向的空间，指针指向的地址不发生变化，但再对指针进行解引用操作时会产生错误
- 若消亡动态变量后程序仍未结束，则将指针变量设为空指针是程序设计的良好习惯

7.4.4 内存泄漏

- 运行中内存泄漏：申请动态空间后，指针变量指向的地址发生变化，导致不再有指针变量指向该地址，因此该地址不再能访问，也不能释放该空间
- 运行结束内存泄漏：程序结束后某些动态变量的空间未释放
- 避免内存泄漏：检查每个 `new` 操作是否都对应了 `delete` 语句

7.5 字符串再讨论

- 直接用字符串常量赋值指针变量时，要求指针变量必须为 `const` 型，例如

```
const char *ptr = "Hello World!";
```

此时使用 `ptr[<index>]` 可以访问字符串中的单个字符，但不能修改字符串中的字符

实质：在内存中开辟空间，存入字符串 "Hello World!\0"，并将起始地址赋值给 `ptr`

- 字符指针与字符数组：

```
char ch[14] = "Hello, World!";  
char *ptr = ch;
```

此时字符指针变量 `ptr` 指向字符数组 `ch` 的起始地址，且可通过指针变量修改字符数组中的内容

- 直接使用字符指针：

```
char *ptr = new char[10];
```

在内存中分配了 10 个 `sizeof(char)` 字节长度的空间并将首地址赋值给 `ptr`

- 函数参数问题：字符数组名和字符指针变量传参的效果等价，因为都是传递字符串的起始地址，且无需指定长度，因为字符串以 `'\0'` 结尾

7.6 引用类型

7.6.1 引用的定义及应用

- 定义：给变量取别名，以使同一地址上的内容可被多个变量访问
实质上是隐式指针，但使用时不写解引用运算符 *

- 变量引用：<type> &<name>=<variable>;

常量引用：const <type> &<name>=<variable>;

- <type> 是类型名
- <name> 是引用类型变量名
- <variable> 是原变量名

使用变量引用时，可通过引用变量访问并修改对应地址空间上的内容

使用常量引用时，只能通过引用变量访问对应地址空间上的内容，不能修改

7.6.2 函数参数的引用传递

- 引用最常用的用途就是传递函数参数
- 例如交换同类型参数的值的函数可写作

```
void swap(int &a, int &b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

调用时可直接写作 swap(x, y);

因为引用传递实际上也是传递地址，形式参数与实际参数对应的是同一地址空间，所以函数能够修改调用处程序的参数值；此外，由于引用传递的性质，实际参数在函数中仍可作为普通变量使用

引用传递要求实际参数必须为左值，因为右值没有对应的存储空间

- 引用传递的优势：减少函数调用的开销（无需为形式参数分配内存空间；引用传递过程中不会复制参数内容）；能修改调用处程序的变量值
- 可使用常量引用传递，避免函数修改调用处程序的变量值，此时形式参数在函数中为常量，例如取最大值函数的原型可声明为

```
int max(const int &a, const int &b);
```

7.6.3 返回引用的函数

- 函数的返回值可以是引用，引用函数中的变量
但不能引用函数执行结束后会消亡的变量，例如函数内部的自动局部变量
- 返回引用的函数原型声明：<type>& <name>(<param>);
 - <type> 是返回引用的基类型
 - <name> 是函数名
 - <param> 是参数列表
- 返回引用的函数可作为左值，因为引用变量有其内存映像

7.7 指针数组与多级指针

7.7.1 指针数组

- 指针数组即元素全为指针的数组
 - 格式：<type> *<name>[<size>;]
 - <type> 是指针元素的基类型
 - <name> 是数组名
 - <size> 是数组长度
- 此时数组中的每个元素都可当作指针使用

7.7.2 二维数组与指针

- 二维数组的内存映像实际上是每个元素均为数组的数组，且顺序存放
基类型与二维数组元素相同的指针可类似地加减移动以指向二维数组
- 指向一维数组的指针：<type> (*<name>)[<size>;]
 - <type> 是基类型
 - <name> 是指针变量名
 - <size> 是数组长度
 - 圆括号()不能省略，否则会变为声明指针数组

7.7.3 多级指针

- 如果指针变量指向的地址上保存的是其他指针变量，称为多级指针，例如

```
int x=8;
int *p=&x;
int **q=&p;
```

此时整型指针变量 p 中保存了整型变量 x 的地址，整型二级指针变量 q 中保存了整型指针变量 p 的地址

8 结构体

8.1 结构体类型定义

- 结构体：将数据类型不同的分量整合为整体，用新的数据类型表示
每个分量称为数据成员，一类结构体中的实例称为对象
- 格式：`struct <class>{<member>;}`;
 - `struct` 是结构体关键字
 - `<class>` 是结构体名，要符合标识符命名规范
 - `<member>` 是成员列表，可以是任意数据类型，也可以是其他结构体，但不能是结构体本身
 - 数据成员的定义与变量定义格式相同

8.2 结构体对象定义

- `<class> <object>;` 定义单个对象
- `<class> <object>[<size>;]` 定义对象数组
- `<class> *<object ptr>;` 定义对象指针
- `struct <class>{<member>} <object>;` 定义结构体同时定义对象
- `struct {<member>} <object>;` 省略结构体类型名直接定义对象，但无法再定义此类结构体的对象
 - `<class>` 是结构体类型名
 - `<object>` 是对象名
 - `<size>` 是数组长度
 - `<member>` 是成员列表

8.3 结构体变量使用

- 初始化：`<class> <object>={<value 1>,<value 2>,<value 3>,...,<value n>}`
依次对应结构体类型定义时的成员列表
- 访问数据成员：`<object>.<member>`
.`是成员运算符，当成员是结构体时可以嵌套`
- 结构体变量的输入：`cin >> <object>.<member>` 依次输入每个数据成员
- 赋值：
 - 对每个数据成员赋值 `<object>.<member> = <value>;`
 - 同类型对象之间相互赋值 `<object 1> = <object 2>;`
- 指针对象的访问：

- 由于成员运算符 `.` 的优先级高于解引用运算符 `*`，因此 `*<object ptr>.<member>` 等价于 `*(<object ptr>.<member>)`
- 要访问结构体指针的数据成员，则要使用 `(*<object ptr>).<member>`
- 为方便起见，C++规定了 `<object ptr> -> <member>` 的写法
等价于 `(*<object ptr>).<member>`

8.4 结构体作为函数参数

- 结构体作为函数参数的最佳方式为引用传递
- 由于结构体空间庞大，值传递会降低程序时空效率
- 指针传递形式复杂，引用传递形式更简洁
- 如果不想在函数体中修改结构体内容，可以用 `const` 限定

9 模块化开发

- 思想：自顶向下，分步求解，将规模较大的问题或应用分解为规模较小的内容，将较小的内容封装为函数，解决大规模问题时调用函数即可
- 形式：封装使用细节，提供使用接口，可单独使用、升级，可多次重用，可多人协作
- 头文件设计：可将常量定义、结构体定义、函数原型声明写在头文件 `.h` 或 `.hpp` 中
但当不同的库将同一个头文件包含，之后又有程序使用了多个包含相同头文件的库时，编译器会认为标识符多次定义而报错
为避免这种情况，需要用特殊的预编译指令（头文件保护符）

```
#ifndef <label>
...
#endif
```

若 `<label>` 未被定义，则执行 `#ifndef` 到 `#endif` 之间的语句，否则跳过中间的语句
所以头文件通常有如下结构：

```
#ifndef <label>
#define <label>
...
#endif
```

- 库设计：库包括库的接口和库函数实现
 - 库的接口即头文件，需要告诉使用者
 - * 库函数的原型
 - * 库中全局变量的声明
 - * 库函数的常量和自定义类型
 - 库的实现即源文件，写在 `.cpp` 文件中
 - * 源文件与头文件一般同名
 - * 源文件需要包含头文件，以便编译器检查函数定义和声明的一致性
 - * 适当注释以便告知使用者功能

常用的注释格式如下（写在函数或自定义类型之前）：

```
/**
 * @brief
 * @param
 * @return
 */
```

分别说明函数、函数参数、函数返回值的含义

- 库的使用：只需要包含 `.h` 或 `.hpp` 文件，不需要包含源文件

10 创建新的类型

10.1 面向对象程序设计

10.1.1 抽象的过程

- 面向过程程序设计：程序从入口处开始执行，到主程序返回时结束
- 面向对象程序设计：程序员根据解决问题的需求，创建新的类型作为工具，再使用这些工具解决问题

例如将圆作为数据类型，面向对象程序设计者相应设计工具，使用者只需知道工具如何使用，就能得到圆的半径、周长、面积等参数

10.1.2 面向对象程序设计的特点

- 代码重用：在设计出面向对象的工具后，不仅可以自己使用，也可供他人使用
- 实现隐藏：新的类型的使用者无需知道该类型是如何实现的，只需知道如何使用
- 更完善的面向对象机制，是从 C 到 C++ 的本质进步

10.1.3 库与类

- 库是程序的源代码文件，类是更加合理的库的内容
- 我们已经知道可以用结构体 `struct` 创建新的数据类型，面向对象程序设计者需要设计相应的函数，以使用结构体这种新的数据类型
- 以结构体作为参数的函数存在以下问题：
 - 难以使用：每次使用都要传递结构体
 - 容易冲突：在函数名称表示函数功能的良好习惯基础上，不同类型很容易拥有重名函数，例如初始化函数 `initialize` 和清空函数 `clear`
 - 工作繁琐：若新的类型中有动态变量，则使用前，使用者需要显式申请空间；使用结束后，使用者需要显式释放空间；同时，函数形式也不支持使用运算符
- 为解决上述问题，C++允许新的类型的设计者将函数放入结构体中，从而使得结构体本身可作为函数的自带参数，也解决了重名问题

例如将圆作为新的类型：

```
// File: circle.h
#ifndef circle_h
#define circle_h

const double pi=3.1415927;

struct circle {
```

```
    double r = 0;
    void change_radius(double new_r);
    double get_radius();
    double get_perimeter();
    double get_area();
};

#endif
```

新类型的函数在类外实现时，用域运算符 `::` 区分，而结构体自动作为函数的自带参数，无需显式声明参数，例如上述结构体内的函数实现：

```
// File: circle.cpp
#include "circle.h"

extern const double pi;

void circle::change_radius(double new_r) {
    r = new_r;
}

double circle::get_radius() {
    return r;
}

double circle::get_perimeter() {
    return 2*pi*r;
}

double circle::get_area() {
    return pi*r*r;
}
```

新类型的函数也可在类内实现，格式与独立的函数相同，类内实现的函数在调用时将自动作为内联函数编译

结构体内的函数可视为结构体的成员，调用时用成员运算符 `.` 即可，例如：

```
// File: main.cpp
#include <iostream>
#include "circle.h"
using namespace std;
```

```
int main() {  
    double x=2.0;  
    circle C;  
    C.change_radius(x);  
    cout << C.get_radius() << endl;  
    cout << C.get_perimeter() << endl;  
    cout << C.get_area() << endl;  
    return 0;  
}
```

- 将函数放入结构体解决了传参和重名问题，但实现细节的隐藏不够彻底，而且没有解决工作繁琐的问题
- 为了让程序设计者能够实现更加安全、更加完善的新的类型，C++提供了全新的定义方法——类

10.2 类的定义

- 格式:

```
class <name> {  
private:  
    <private_member>  
public:  
    <public_member>  
};
```

- class 是类声明关键字
- private 和 public 是访问控制关键字
- <private_member> 是私有成员
- <public_member> 是公有成员
- 成员: 包括数据成员和成员函数
- 私有成员和公有成员:
 - 私有成员只能被类的其他成员访问
 - 公有成员可由类内外所有对象访问，是类对外的接口
- 实际上，在 C++ 中使用 struct 也可类似地定义新的类型
- private 和 public 出现次数和出现顺序都可任意，也允许部分成员不声明访问控制
 - 当类使用 struct 定义时，不声明访问权限的成员默认为 public
 - 当类使用 class 定义时，不声明访问权限的成员默认为 private

- 类的成员函数的实现和使用均与用 `struct` 实现的类相同

10.3 对象的使用

10.3.1 对象的定义

- 格式: `<storage> <class> <object>;`
 - `<storage>` 是存储类别
 - `<class>` 是类名
 - `<object>` 是对象名
 - 分别对应内置数据类型的存储类别、数据类型、变量名也可类似地定义指针、数组，申请动态变量，释放动态变量等

10.3.2 对象的操作

- 访问成员: `<object>.<member>` 或 `<object ptr> -> <member>`
 - `<object>` 是对象名, `<object ptr>` 是对象指针
 - `<member>` 是成员, 可以是数据成员和成员函数 (调用函数时要按要求传递参数)
 - 外部函数只能访问类的公有成员
- 赋值: `<object 1> = <object 2>`

实质上是从 `<object 2>` 的存储空间, 取所有数据成员的值, 复制给 `<object 1>` 的对应数据成员

10.3.3 this指针

- 为了优化内存结构, 每个类的成员函数只会在内存中保留1份
- `this` 指针: 调用成员函数时, 用于确定调用函数的对象
 - 作为成员函数的参数, 使得函数中出现的数据成员默认是调用函数的对象的数据成员 (即等价于在数据成员前面加上 `this->`)
- 当成员函数的形式参数与类的数据成员同名时, `this` 指针必须显式使用

10.4 对象的构造与析构

10.4.1 构造函数

- 构造函数: 在定义对象时自动调用的函数
- 构造函数的函数名与类同名, 不可以声明返回类型 (声明为 `void` 也不可以), 参数列表可根据需求指定, 也可以没有参数
- 同个类可以有参数列表不同的不同构造函数

- 若未定义构造函数，编译器将生成隐式的默认构造函数
默认构造函数的参数列表和函数体均为空
- 定义构造函数后，对象的声明格式变为 `<class> <object>(<param>);`
动态对象申请的格式变为 `new <class>(<param>)`
 - `<class>` 是类名
 - `<object>` 是对象名，可在其前添加 `*` 表示指针型变量
 - `<param>` 是构造函数的实际参数表
 - 仅当该类定义了无形式参数的构造函数，或该类定义了形式参数均有默认值的构造函数时，`(<param>)` 部分可以省略，此时定义对象将调用相应的默认构造函数或自定义构造函数

10.4.2 初始化列表

- 格式: `<name>(<param>):<member 1>(<param 1>),<member 2>(<param 2>),...,<member n>(<param n>);`
 - `<name>` 为构造函数名
 - `<param>` 是参数列表
 - `<member i>` 是类的数据成员
 - `<param i>` 来自参数列表，是函数的形式参数
- 执行: 定义初始化列表后，执行构造函数时，首先用实际参数初始化数据成员，然后执行函数体的内容
- 优势: 数据成员的定义和初始化同时进行，有利于提高代码效率；同时还可完成常量的初始化（常量不能只能初始化时赋初值）

实际上，即使不使用初始化列表，而是在构造函数的函数体内完成数据成员初始化，程序也会在执行构造函数的函数体前，调用数据成员子类的构造函数，这在类的组合与继承⁷中对代码效率是极为不利的，而且可能导致访问权限不足的问题

10.4.3 析构函数

- 析构函数: 在对象消亡时自动调用的函数
- 析构函数的函数名为类名之前添加 `~`，**不可以声明返回类型**（声明为 `void` 也不可以），**也不能有参数**
- 若未定义析构函数，编译器将生成隐式的默认析构函数
默认析构函数的参数列表和函数体均为空
- 当类中有动态变量时，自定义析构函数并在其中为动态变量释放空间是必要的，否则会造成内存泄漏问题

⁷有关组合与继承的更多内容，参见第12章

10.4.4 复制构造函数

- 复制构造函数：利用同类的其他对象构造新对象的函数
- 格式：`<class>(const <class> &);`
 - `<class>` 是类名，也是构造函数名
 - 复制构造函数以其同类对象的引用作为参数
- 若未定义复制构造函数，编译器将生成隐式的默认复制构造函数
默认复制构造函数会根据其他对象，依次为数据成员赋值
当数据成员包含指针变量时，默认的复制构造函数会让两个对象的指针数据成员指向同一空间，这在使用和析构的过程中可能出现问题
- 复制构造函数调用的情况：
 - 用同类的其他对象初始化新对象时
 - * 直接在对象定义时用赋值运算符或小括号初始化，此时将跳过构造函数，直接调用复制构造函数
 - * 对象定义后，在其后的语句中赋值时，将先后执行构造函数和赋值运算符重载函数⁸
 - 函数调用过程中，采用值传递方式将对象传递给函数时
 - 函数结束后，把对象作为返回值时

10.5 常量与类

10.5.1 常量数据成员

- 类的数据成员可用 `const` 定义为常量
- 同类的不同对象的常量数据成员可以不同，但在特定对象的整个生命周期内，常量数据成员的值不可以改变
- 常量数据成员只能而且必须在构造函数的初始化列表中赋初值，不可在常量数据成员定义时赋初值，也不可以在构造函数的函数体中赋值，更不能在其他成员函数中修改常量数据成员的值

10.5.2 常量对象

- 常量对象与内置数据类型的常量形式相同
 - 其构造函数必须含有全部数据成员的初始化列表
 - 只能而且必须在定义对象时赋初值
 - 试图修改常量对象时，程序会报错

⁸有关运算符重载的更多内容，详见第11章

10.5.3 常量成员函数

- 常量对象只能调用常量成员函数
- 格式: `<type> <name>(<param>) const;`, 即在参数列表之后添加 `const`
实现常量成员函数时, 也必须在参数列表之后、函数体之前添加 `const`, 否则会被编译器视为不同函数
- 常量成员函数中不允许:
 - 修改数据成员的值
 - 调用其他非常量成员函数
- 不修改数据成员的函数定义为常量成员函数是良好的编程习惯

10.6 静态成员

10.6.1 静态数据成员

- 静态数据成员不属于对象, 而是属于整个类, 内存映像位于全局变量区
定义对象时, 并不会为静态数据成员分配空间
静态数据成员的空间分配是在编译时完成的, 因此类的实现必须为静态数据成员赋初值, 且只能分配一次
- 格式: 在类的数据成员定义前添加 `static`
- 实现: `<type> <class>::<variable> = <value>;`
 - `<type>` 是数据类型
 - `<class>` 是类名
 - `<variable>` 是静态数据成员名
 - `<value>` 是静态数据成员的初值该语句为类的静态数据成员分配空间, 并赋初值
- 使用: `<class>::<variable>` 或 `<object>.<variable>`
 - `<class>` 是类名
 - `<object>` 是对象名
 - `<variable>` 是静态数据成员名即可以通过类直接访问静态数据成员, 也可以通过对象访问静态数据成员, 但无论通过何种方式访问, 指向的都是同一块空间

10.6.2 静态成员函数

- 静态成员函数为类的全体对象服务, 而不是为某个特定对象服务, 一般用于操作静态数据成员

- 格式：在函数原型声明之前添加 `static`
在类外实现静态成员函数时，不写 `static`
- 与一般成员函数的不同之处：不包含 `this` 指针，无法使用非静态的数据成员，因为静态成员函数是属于整个类，而不是属于特定对象

10.6.3 静态常量数据成员

- 静态常量数据成员是整个类的常量数据成员
- 格式：在类的数据成员定义前添加 `static const`
- 只能而且必须在类的定义时初始化

10.7 友元

- 友元：类可以允许程序的其他部分访问自己的私有成员
 - 友元的类型分为：
 - 友元函数（某个全局函数）
 - 友元类（其他类）
 - 友元成员函数（其他类的某个成员函数）
 - 友元是通过授予获得的，而不是通过索取获得的，例如类 A 可声明函数 f 是自己的友元，而不能函数 f 声明自己是类 A 的友元
 - 友元是非对称的，即类 A 是类 B 的友元并不意味着类 B 是类 A 的友元
 - 友元是非传递的，即类 A 是类 B 的友元，且类 B 是类 C 的友元，并不意味着类 A 是类 C 的友元
 - 格式：用友元关键字 `friend` 声明友元
 - 声明友元函数时，在函数原型声明之前添加 `friend`
 - 声明友元类时，在类名之前添加 `class` 之后，再在最前面添加 `friend`
 - 声明友元成员函数时，需先声明其他类，之后在本类中声明友元，涉及其他函数的原型时要添加类名和域运算符
- 必须在类内部声明，可在公有部分声明也可在私有部分声明
可在类内或类外实现，类外实现时不需要再次用友元关键字 `friend` 声明

11 运算符重载

11.1 运算符重载的意义

- 优势：使系统内置的运算符可作用于类的对象，使自定义类在使用上更接近内置数据类型
- 局限：部分运算符不允许重载，例如
 - 成员运算符 `.`
 - 解引用运算符 `*`
 - 域运算符 `::`
 - 选择运算符 `?` 和 `:`
 - 占用字节数运算符 `sizeof()`
- 不能创建新的运算符
- 不能改变运算符的操作数个数
- 不能改变运算符的优先级和结合性

11.2 运算符重载的格式

- 运算符通过函数的形式重载，函数名为 `operator@`，并将 `@` 替换为想要重载的运算符
- 作用域：大部分运算符都能重载为成员函数或全局函数
 - 重载为全局函数时，形式参数个数与运算符的原操作数个数相同，一般在类中设为友元函数
 - 重载为成员函数时，由于成员函数含有隐式的 `this` 指针作为参数，因此形式参数的个数比对应的全局函数少1个
- 成员函数和全局函数的选择：
 - 必须重载为成员函数：`[]` 和 `->`
 - 最好重载为成员函数：`++` 和 `--` 等
 - 重载为全局函数时更加灵活：可以与其他类型的变量或对象运算
- 二元运算符重载格式：
 - 成员函数：`<type> operator@ (<class> &<object>);`
 - 全局函数：`<type> operator@ (<class 1> &<obj 1>, <class 2> &<obj 2>);`
 - * `<type>` 是返回类型
 - * `@` 是重载的运算符
 - * `<class i>` 是操作数的类
 - * `<obj i>` 是操作数的对象，一般采用引用传递

11.3 特殊运算符的重载

11.3.1 赋值运算符重载

- 复制构造函数和赋值运算符重载函数的区别：
 - 复制构造函数只需要负责新对象的创建
赋值运算符重载函数需要负责原有空间的释放和新对象的创建
 - 复制构造函数没有返回值
赋值运算符重载函数返回引用，因此运行能够连续赋值
 - 复制构造函数在目标对象未构造时调用
赋值运算符重载函数在目标对象已构造时调用
 - 复制构造函数无需检查自赋值
赋值运算符重载函数一般需要检查是否是对象为对象本身赋值
例如赋值运算符重载函数涉及空间释放，而且没有自赋值检查时，执行 `a=a;` 会导致对象 `a` 内容丢失
自赋值检查的形式为 `(this != &<object>)`
- 赋值运算符必须重载为成员函数，形式参数是同类对象的引用，必须返回同类对象的引用
- 格式: `<class>& operator= (const <class> &<object>);`
 - `<class>` 是类名
 - `<object>` 是同类对象名，采用引用传递

11.3.2 下标运算符重载

- 下标运算符必须重载为成员函数，形式参数是下标，返回数组元素的引用
- 格式: `<type>& operator[] (int <index>);`
 - `<type>` 是返回类型
 - `<index>` 是下标
- 由于返回值是数组元素的引用，因此既可以作为左值又可以作为右值

11.3.3 函数调用运算符重载

- 函数调用运算符 `()` 是二元运算符，第一个运算数是函数名，第二个运算数是形式参数表，返回函数的返回值
- 函数调用运算符必须重载为成员函数
- 格式: `<type> operator() (<param>);`
 - `<type>` 是返回类型
 - `<param>` 是参数列表

- 重载函数调用运算符后，该类的对象就可以直接作为函数使用
格式为 `<object>(<param>);`

11.3.4 自增和自减运算符重载

- 最大的问题：既有前缀又有后缀，而且参数列表完全相同
- 解决方法：设为不同的重载函数，将后缀运算视为二元运算符
- 格式：
 - 前缀成员函数：`<class>& operator@@ ();`
 - 后缀成员函数：`<class> operator@@ (int);`
 - 前缀全局函数：`<class>& operator@@ (<class> &<object>);`
 - 后缀全局函数：`<class> operator@@ (<class> &<object>, int);`
 - * `<class>` 是返回类名和自增/自减的对象所属类名
 - * `@` 替换为 `+` 或 `-`，分别重载自增和自减函数
 - * `<object>` 是自增/自减的形式参数对象名
 - * 后缀自增/自减函数的重载函数参数列表中的 `int` 仅表示重载的是后缀运算符，而不是整型变量作为形式参数

11.3.5 流运算符重载

- 流插入运算符 `<<` 是二元运算符，第一个运算数是输出流类 `ostream` 类的对象，第二个运算符是内置数据类型的变量或用户自定义的类型的对象，返回类型是输出流类对象的引用（因此可以连续输出），结合性为左结合
- 流提取运算符 `>>` 是二元运算符，第一个运算数是输入流类 `istream` 类的对象，第二个运算数是内置数据类型的变量或用户自定义的类型的对象，返回类型是输入流类对象的引用（因此可以连续输入），结合性为右结合
- 格式：
 - `ostream& operator<< (ostream& <os_obj>, const <class> &<object>);`
 - `istream& operator<< (istream& <is_obj>, <class> &<object>);`
 - * `ostream` 和 `istream` 分别是输入流类和输出流类，因此重载前必须使用预编译指令包含输入输出流标准库 `<iostream>`
 - * `<os_obj>` 和 `<is_obj>` 分别是输入流类和输出流类的对象，函数的返回值要和对应形式参数相同，以加速连续输入/输出的过程
 - * `<class>` 是类名，`<object>` 是同类对象名，采用引用传递
 - * 由于输出不改变对象而输入改变对象，因此流插入运算符重载时采用常量引用传递，流提取运算符重载时采用变量引用传递（流提取运算符重载时必须采用引用传递，否则不能将输入存放在正确的位置）

11.4 自定义类型转换函数

11.4.1 内置类型数据转换为类的对象

- 通过类的构造函数实现，只要类中有参数列表符合要求的构造函数，即参数列表只有 1 个内置类型的参数，在需要隐式转换的场合，程序便会以实际参数构造对象，并进行赋值或比较等
- 避免：如果不希望以上过程执行，可在对应构造函数声明前添加关键字 `explicit`，此时再遇到需要需要隐式转换的场合时，编译器会报错

11.4.2 类的对象转换为其他类型

- 使用类型转换函数：必须定义为成员函数
- 格式 `operator <class>() const;`
 - `<class>` 是目标类型名
 - 函数返回值必须为目标类型的对象或能转换为目标类型的表达式

12 组合与继承

12.1 组合

- 概述：将某个类的对象作为其他类的数据成员，称为对象成员；是将类与内置数据类型或类与类组合为新类的过程；初始的类与新的类是部分与整体的关系
- 新类的成员函数不能访问其对象成员的私有数据成员和成员函数
- 初始化：由于大多数对象成员的数据成员是私有的，必须调用对象成员的构造函数，所以新类的构造函数往往有初始化列表，通过初始化列表调用对象成员的构造函数，以完成对象成员的初始化

若不指定初始化列表，则会调用对象成员的默认构造函数；若对象成员没有可用的默认构造函数，则编译器会报错

12.2 继承

- 概述：在已有类的基础上，添加其他功能以实现功能更复杂、更全面的类；是继承类的已有功能创建新类的过程；初始的类与新的类是一般与特殊的关系
- 已有的类称为基类，新创建的类称为派生类

12.2.1 派生类的定义

- 保护访问特性：不能被全局函数和其他类的成员函数访问，但可以被派生类的成员函数和派生类的友元函数访问
用关键字 `protected` 声明，与 `private` 和 `public` 并列
- 继承方式：决定基类的数据成员在派生类中的访问特性
 - `public` 继承：基类成员的访问特性在派生类中不变
 - `protected` 继承：基类的 `private` 成员在派生类中均变为 `protected`，其他访问特性不变
 - `private` 继承：基类成员的访问特性在派生类中均变为 `private`
- 格式：`class <name>:<method> <basic> {<new_member>;}`
 - `<name>` 是派生类名
 - `<method>` 是继承方式
 - `<basic>` 是基类名
 - `<new_member>` 是新成员声明

继承方式通常是 `public`，以保持基类的访问特性
- 派生类访问基类的可访问成员时，不需要用基类名和成员运算符，直接使用基类的成员名即可

12.2.2 派生类对象的构造和析构

- 构造函数：基类数据成员可通过基类构造函数完成初始化，此时需要派生类构造函数调用基类的构造函数：若基类数据成员对派生类可见，则基类数据成员也可在派生类构造函数中完成初始化
- 析构函数：派生类的析构函数函数体被执行完后，程序会自动调用基类的析构函数，无需显式调用

12.2.3 重定义基类的函数

- 允许派生类和基类拥有同名的不同函数
- 需要在派生类的定义中重新声明并定义
- 可在派生类重定义基类的函数中调用基类的同名函数，但要注意使用作用域运算符，表示调用基类的函数而不是递归调用
- 通过派生类对象调用成员函数时，自动调用派生类的函数，基类的同名函数被隐藏

12.2.4 派生类对象与基类对象

- 基类对象不能自动转化为派生类对象，除非在基类中定义了向派生类的类型转换函数；不能将基类对象地址赋给派生类对象指针；不能将基类对象指针赋给派生类对象指针
- 派生类对象可自动转化为基类对象，此时只给基类的数据成员赋值，派生类新增的数据成员自动舍弃
- 基类对象指针可指向派生类对象，此时只能通过该指针访问基类部分的公有成员；基类对象引用同理

12.3 多态性

12.3.1 多态性

- 多态：不同类的对象调用同名的函数时做出不同的动作，便于系统功能的扩展
 - 编译时的多态性：通过重载函数和运算符重载，根据参数列表的不同实现多态
 - 运行时的多态性：通过虚函数和基类指针指向不同派生类函数，直到运行时才决定具体调用的函数实现多态

12.3.2 虚函数

- 格式：类定义中函数原型声明前添加虚函数关键字 `virtual`
- 其派生类可重定义该函数，也可不重定义该函数
- 重定义要求：派生类重定义基类函数的原型（返回类型和参数列表）必须与基类中同名函数的原型一致

- 使用：通过基类指针或基类引用调用虚函数时，检查指向基类对象还是派生类对象
 - 若指向基类对象，则执行基类虚函数
 - 若指向派生类对象，检查派生类是否重定义了虚函数
 - * 若未重定义虚函数，则执行基类虚函数
 - * 若重定义虚函数，则执行派生类虚函数

12.3.3 虚析构函数

- 产生背景：基类指针指向派生类对象时，若调用基类的析构函数，则会造成派生类对象部分内存被释放，此时无论是否调用派生类虚析构函数都会造成内存问题
- 解决方法：将基类析构函数定义为虚函数，并在派生类中重定义析构函数，通过基类指针调用虚析构函数时，程序会先找到派生类的重定义析构函数并调用，之后自动调用基类的析构函数，保证完全释放派生类对象的内存

12.4 纯虚函数与抽象类

- 纯虚函数：在基类中声明但不定义的函数，无法运行，需要派生类重定义
- 格式：在函数原型声明的参数列表之后添加 =0
- 抽象类：含有纯虚函数的类，不能实例化，表现在：
 - 抽象类的指针只能指向派生类对象
 - 不能定义抽象类对象
- 作用：作为基类，通过基类指针实现派生类的多态性，同时防止派生类不定义该函数而影响运行

13 输入输出与文件

- 输入输出指 CPU 和内存与外部设备交换信息的过程
- C++语言将输入输出视为字节的流，输入输出操作由标准库提供，分为基于控制台（命令行终端）的输入输出、基于文件的输入输出、基于字符串的输入输出

13.1 流与标准库

- C++通过字节的流实现输入输出
 - 低级的输入输出：流是字节的组合
 - 高级的输入输出：多个字节组成有意义的单位，例如 4 字节组成整型数据等
- 头文件：
 - `<iostream>` 基于控制台的输入输出
 - `<fstream>` 基于文件的输入输出
 - `<sstream>` 基于字符串的输入输出

13.2 输入输出与缓冲区

- 控制台输入输出基于缓冲区实现
- 用户输入内容并输入 Enter 后，输入的内容与 Enter 储存在缓冲区中，当执行流提取运算符时，程序从缓冲区读取内容并存入变量；若缓冲区为空，程序会暂停等待用户输入
- 使用流插入运算符后，程序会将内容写入命令行终端或文件

13.3 基于控制台的输入输出

- 标准库与流对象：
 - `cin` 是输入流类 `istream` 的对象
 - `cout` 是输出流类 `ostream` 的对象
- `cout` 的格式化输出与无格式输出：
 - 流插入运算符 `<<` 对应标准输出，根据数据类型解释内存空间上的内容并输出，其中 `char*` 类型会自动识别为字符指针，并输出其指向地址上的字符串，如需输出 `char*` 的实际地址需要强制类型转换
 - 成员函数 `put` 对应字符输出，将参数按其字节的内容转化为 ASCII 码表中对应的字符依次输出
 - 成员函数 `write` 对应无格式输出，以字符形式输出从某个地址开始的若干个字节
- `cin` 与输入流：

- 流提取运算符 `>>` 根据变量类型进行不同方法的读入，会跳过空白字符（空格、`'\t'`、`'\n'`等）
 - 一般情况下，`cin` 与变量或对象的 `>>` 运算会返回 `cin`，但遇到 `Ctrl+Z` 或者文件的终止符时，会返回 `EOF` 表示输入结束，此后的 `>>` 与变量或对象等全部失效
 - 此外，`EOF` 转换为 `bool` 型变量的结果是 `false`，`cin` 转换为 `bool` 型变量的结果是 `true`，可作为分支结构或循环结构的判断条件
- 成员函数 `get` 从缓冲区接受 1 个字符（包括空白字符）并返回
- 成员函数 `getline` 从缓冲区最多接受参数“最大长度”个字符，直到遇到参数“分隔符”或者达到最大长度，存入参数“目标地址”
 - 目标地址后面预留的空间要足够大；该函数不一定读取到最大长度；除非分隔符是空格，否则不会跳过空格；达到最大长度后函数必然退出；分隔符的默认值是 `'\n'`
- 成员函数 `read` 从缓冲区读取一定长度的字节并存放放到目标位置，不会检查字节的含义，也不会写入 `'\0'`，直到读满字节数（除非遇到 `Ctrl+Z` 或文件的终止符）
- 格式化输入输出：位于标准库 `<iomanip>` 中，提供格式化输入输出的流操纵运算数和函数
 - 设置整型数基数：
 - * 流操纵运算符：`oct` 表示八进制，`dec` 表示十进制，`hex` 表示十六进制
 - * 成员函数：`setbase(int x);`，其中 `x` 为 8,10,16
 - 设置浮点数精度：
 - * 流操纵运算符：`setprecision(int i);`，`i` 表示小数点后 `i` 位
 - * 成员函数：`precision(int i);`，`i` 表示小数点后 `i` 位
 - * 有效范围是设置后到下一次设置前
 - 设置作用域宽：
 - * 流操纵运算符：`setw(int i);`，`i` 表示字符数
 - * 成员函数：`width(int i);`，`i` 表示字符数
 - * 有效范围是只生效 1 次
 - * 用于输入时，只读取 `i-1` 个字符到变量
 - * 用于输出时，若输出的字符数小于 `i` 则用空格填充

13.4 基于文件的输入输出

- 在外部存储中用以永久保存数据的信息集合称为文件，以文件名和文件路径为标识符
- C++语言中的文件：视为字节数组（有序的字节流，以 `EOF` 结束）
- 文件操作流程：

1. 定义文件流对象，可用的类有：
 - 文件输入流 `ifstream` 类，只能读
 - 文件输出流 `ofstream` 类，只能写
 - 文件操纵流 `fstream` 类，可读可写
2. 打开文件：`open` 成员函数，需要以参数指定打开模式：
 - `in` 读取文件
 - `out` 写入文件
 - `app` 写入前定位到文件末尾
 - `ate` 打开后定位到文件末尾
 - `trunc` 打开后清空文件
 - `binary` 以二进制模式输入输出
3. 操纵文件中的数据
4. 关闭文件：`close` 成员函数，没有参数
显式关闭文件是良好的编程习惯

14 异常

- 传统异常处理：检测到错误时就地解决
 - 优点：便于直接看到错误处理情况
 - 缺点：错误处理的代码插入程序的逻辑，影响程序的可读性
- C++的异常机制：程序遇到抛出异常时停止正常代码执行，寻找异常捕获代码
 - 异常抛出：`throw <object>;`，其中 `<object>` 可以是变量或对象，或者结果为变量或对象的表达式
 - 异常捕获的启动：`try {<codeblock>}`，程序在 `{<codeblock>}` 中触发异常抛出后会直接跳出 `try` 语句块
 - 异常处理：`catch (<class> <param>) {<codeblock>}`，程序在捕获异常后会寻找 `catch` 语句块，若 `throw` 抛出的变量或对象的类型与 `<class>` 匹配，则执行 `<codeblock>` 语句块中的内容；`<param>` 可以作为 `<codeblock>` 的形式参数，若不需要使用则可以省略
 - 也可以用 `catch (...)` 表示捕获任意异常